

Task Decomposition

1. What is task decomposition?

When a goal is too large for a single agent pass, you break it into subtasks. The critical design decision is whether those subtasks run in parallel or in sequence. Getting this wrong wastes time, breaks pipelines, or produces incorrect results. The exam tests your ability to look at a scenario and immediately identify the correct strategy.

2. The three decomposition strategies

Parallel Decomposition

Subtasks are independent — no subtask needs another's output to start.

- All subtasks fan out and run simultaneously.
- Results are collected and merged after all complete.
- Best for: searching multiple sources, processing multiple files, running multiple evaluations.
- Exam trigger words: 'simultaneously', 'independent', 'multiple sources', 'fan out', 'batch'.
- Watch out for: false independence — tasks that look parallel but share state or need ordering.

Sequential Decomposition

Each step has a hard dependency on the previous output.

- Subtasks run one after another — each stage waits for the previous to complete.
- Best for: ETL pipelines, multi-stage validation, document processing chains.
- Exam trigger words: 'depends on', 'requires output of', 'pipeline', 'stages', 'ETL', 'in order'.
- Watch out for: unnecessary sequencing — don't force order on tasks that could safely run in parallel.

Hybrid Decomposition
<i>Sequential stages where some stages internally fan out into parallel subtasks.</i>
• Most real production systems are hybrid. • Some stages must be sequential (hard dependencies between stages). • Within a stage, independent subtasks can fan out in parallel. • Example: gather requirements (sequential) → research 5 topics in parallel → merge (sequential) → write report (sequential). • Exam trigger: 'within each stage', 'for each item in the list', mixed dependencies.

3. The dependency test

Before choosing parallel or sequential, ask these three questions about every subtask pair. If ANY apply → sequential. If NONE apply → parallel is safe.

Dependence Type	Question to ask	Example	Fix
Data	Does B need A's output to begin?	<i>Clean data before analysing</i>	Sequential
State	Do both read/write the same shared resource?	<i>Two agents writing the same config file</i>	Sequential or locking
Order	Does meaning change based on execution order?	<i>Step 1 must set context for step 2</i>	Sequential

4. Granularity — a common exam trap

Too coarse
• Subtasks are too large — context window fills up mid-task. • Agents cannot complete their portion cleanly. • Fix: split subtasks into smaller, more focused units.

Too fine

- Overhead from coordinating dozens of tiny subtasks exceeds the parallelism benefit.
- Token costs and latency explode with too many agent invocations.
- Fix: batch related items — e.g. 10 agents processing 20 contracts each instead of 200 agents processing 1 each.

Just right

- Each subtask fits cleanly in one agent's context window.
- Each subtask produces a self-contained output the orchestrator can act on directly.
- Coordination overhead is small relative to the parallelism benefit.

5. Result merging strategies

Parallel decomposition always creates a merging problem. After all subtasks complete, someone must combine the results. The exam tests two merge strategies:

Strategy	How it works	Use when
Concatenation	Simply join results together in order.	Outputs are independent and non-overlapping. E.g. search results from different sources.
Synthesis merge	A dedicated merge agent reads all results and produces unified output.	Outputs overlap or contradict each other. Required for coherent, deduplicated, synthesised content.

6. Exam-critical comparison table

Dimension	Parallel	Sequential	Hybrid
Dependency	None between subtasks	Hard — each needs previous output	Mixed — some stages sequential, within-stage parallel
Speed	Fast	Slow	Optimised
Failure model	One subtask fails, others continue	One stage fails, pipeline halts	Depends on which stage fails
Merge needed?	Yes — always	No — output flows naturally	Yes — at merge points

Exam trigger	"independent", "batch", "fan out"	"depends on", "pipeline", "ETL"	"within each stage", "for each item"
--------------	--------------------------------------	------------------------------------	---

7. Quick recall — 3 things to remember

- 1 The dependency test is your first move.**
 Data dependency, state dependency, or order dependency → sequential. None of the three → parallel.
- 2 Granularity matters.**
 Too coarse = context overflow. Too fine = coordination overhead. Sweet spot = one clean output per subtask.
- 3 Parallel always needs a merge step.**
 Choose concatenation for independent outputs, synthesis merge for overlapping or contradicting ones.

8. Practice questions & answers

Three questions from today's session. Q1 was answered incorrectly — the hybrid pattern trap is one of the most common on the exam. Review carefully.

Q1. You are building a Claude system to generate a market research report. The workflow is: (1) Search 5 different industry sources simultaneously, (2) Merge all search results into a unified findings document, (3) Write the final report based on the merged findings. How should this be decomposed?

A) Fully sequential — search source 1, then 2, then 3, then 4, then 5, then merge, then write

B) Fully parallel — all three stages run simultaneously

✓ C) Hybrid — the 5 searches run in parallel, then merge and write run sequentially

D) Single agent — load all 5 search tools into one agent and run them one by one

Explanation:

- C is correct: The 5 searches are completely independent — no search needs another's output to start, so they fan out in parallel. But the merge step has a hard data dependency on ALL 5 search results, and the write step depends on the merged document. Those must be sequential. Parallel where you can, sequential where you must — that's the hybrid pattern.
- A is wrong (the trap): Searching sequentially works but is unnecessarily slow. No data dependency exists between the 5 searches. Forcing sequential order on independent tasks is called unnecessary sequencing — the exam specifically tests whether you can spot and avoid this.
- B is wrong: You can't run the merge while searches are still running — hard data dependency.
- D is wrong: Single-agent with sequential tool calls is functionally the same as A — loses all parallelism.

Q2. A Claude orchestrator is decomposing a document processing task. It receives 200 customer contracts and needs to extract key clauses from each one. What is the biggest risk of decomposing this into 200 parallel subtasks — one per contract?

A) Data dependency violation — contracts may reference each other and must be processed in order

✓ B) Coordination overhead — spawning 200 agents creates token costs and latency that may exceed the parallelism benefit

C) Context overflow — each contract will exceed the subagent's context window

D) Trust boundary violation — subagents cannot be trusted to process sensitive documents

Explanation:

- B is correct: This is the too-fine granularity trap. Spawning 200 individual agents creates massive coordination overhead. The right approach is batching — e.g. 10 agents processing 20 contracts each to balance parallelism against coordination cost.
- A is wrong: Customer contracts are independent documents — no data dependency exists between them.
- C is wrong: One contract per subagent is unlikely to overflow context. Context overflow is a risk when tasks are too coarse, not too fine.
- D is wrong: Trust boundaries are about permissions and prompt injection, not document sensitivity.

Q3. Two parallel subtasks run simultaneously: Subtask A reads a shared config file and updates a setting. Subtask B also reads the same config file and updates a different setting. After both complete, only Subtask B's change is present. Subtask A's change is lost. What went wrong, and what is the correct fix?

A) Data dependency violation — Subtask A should have run before Subtask B sequentially

B) Granularity problem — both tasks should have been merged into one larger subtask

✓ C) **State dependency violation — both subtasks share a writable resource and needed sequential execution or a locking mechanism**

D) Merge strategy error — the orchestrator should have used synthesis merge instead of concatenation

Explanation:

- C is correct: This is a state dependency violation — the most dangerous type because it looks safe on the surface. Both subtasks share a writable resource (the config file). When both read and write simultaneously, the second write overwrites the first — a race condition. Fix: sequential execution or a locking mechanism (mutex).
- A is wrong: Sequential execution would fix it, but the diagnosis is wrong. The issue is shared writable state, not output dependency. Getting the diagnosis right matters on the exam.
- B is wrong: Merging into one subtask would also fix it, but locking is the production-grade answer.
- D is wrong: Merge strategy is about combining results after tasks complete, not preventing write conflicts during execution.

Day 3 scorecard

2 / 3

Good session — one to review

Hybrid decomposition identification	✗ Review
Granularity — too fine granularity risk	✓ Correct
State dependency & race conditions	✓ Correct

Key takeaway from the missed question — spotting hybrid patterns

- When you see a workflow, map each step with this quick check:
- 'Can this step start before the previous one finishes?' → Yes = parallel candidate. No = sequential.

- Hybrid trigger: independent tasks WITHIN a stage + dependent stages BETWEEN stages.
- Classic pattern: search multiple X simultaneously → merge → write. That's hybrid every single time.
- Unnecessary sequencing = forcing order on tasks that could safely run in parallel. The exam tests this.

Next up: Day 4

Session management — handoff patterns and state across turns.